VehicleSure

Vehicle Insurance Management Platform

Software Requirements Specification

Version 1.0 | March 2025

Status: Draft | Classification: Confidential

Backend Module: WillowShield | Frontend: VehicleSure UI

Table of Contents

1. Project Overview 4
2. User Roles & Personas 5
2.1 Customer (ROLE_CUSTOMER) 5
2.2 Underwriter (ROLE_UNDERWRITER) 5
2.3 Claims Officer (ROLE_CLAIMS_OFFICER) 6
2.4 Administrator (ROLE_ADMIN) 6
2.5 Guest (Unauthenticated) 6
3. Functional Requirements 7
3.1 Authentication & Authorization (FR-AUTH) 7
3.2 Quote Generation & Policy Management (FR-POLICY) 7
3.3 Claims Management (FR-CLAIMS) 8
3.4 Administration (FR-ADMIN) 9
3.5 Notifications (FR-NOTIF) 9
4. Non-Functional Requirements 10
4.1 Performance 10
4.2 Security 10
4.3 Scalability 11
4.4 Data Integrity 11
4.5 Usability 11
4.6 Maintainability 11
5. Data Models & Entities 12
5.1 MyUser 12
5.2 Vehicle 12
5.3 VehicleApplication 13
5.4 PolicyPlan 13
5.5 Policy 14
5.6 Payment 14
5.7 Claims 14
5.8 ClaimDocument 15
5.9 ClaimsPayment 15
5.10 Notification 15
5.11 VehicleDocument 16
5.12 Entity Relationship Summary 16
6. API Reference 17
6.1 Authentication APIs 17

6.2 Customer APIs 17
6.3 Payment APIs 18
6.4 Underwriter APIs 18
6.5 Claims Officer APIs 18
6.6 Admin APIs 19
6.7 Notification API 20
7. Workflow & State Machines 21
7.1 VehicleApplication Status Flow 21
7.2 Policy Status Flow 21
7.3 Claim Status Flow 22
7.4 Payment Status Flow 22
8. Third-Party Integrations 23
8.1 Razorpay — Payment Gateway 23
8.2 RazorpayX — Claim Payout 23
8.3 n8n — Email Notifications 24
8.4 Apache POI — Excel Reports 24
8.5 OpenPDF — Document Generation 24
9. System Architecture & Tech Stack 25
9.1 Frontend 25
9.2 Backend 25
9.3 Database 26
9.4 Authentication Mechanism 26
9.5 File Storage 26
9.6 Environment Variables 26
10. Security Requirements 27
11. Constraints & Known Limitations 28
12. Assumptions & Dependencies 29
13. Future Scope 30
14. Glossary 31

1. Project Overview

1.1 Project Identity

Project Name	VehicleSure
Backend Module	WillowShield (Spring Boot Maven artifact)
Type	Full-Stack Enterprise Web Application (B2C + Internal)
Stage	MVP / Beta
Frontend	Angular 21 SPA — localhost:4200
Backend	Spring Boot 3.3.5 REST API — localhost:8080
Database	MySQL 8 — vehiclesure database

1.2 Problem Statement

Traditional vehicle insurance involves excessive paperwork, manual underwriting processes, and slow claim disbursement often taking weeks or even months. Key pain points include:

- Customers have no real-time visibility into their application or claim status.
- Underwriters and claims officers have no centralized assignment or workload tracking tool.
- Claim payouts are manual, slow, and involve physical cheques or offline bank transfers.
- Policy documents and invoices are generated manually and delivered by post.
- There is no digitized premium calculation engine — quotes are ad-hoc and inconsistent.

1.3 Solution Summary

VehicleSure is a full-stack, enterprise-grade vehicle insurance management platform that digitizes and automates the entire insurance lifecycle. The system solves the above problems by:

- Automating premium calculation with a mathematical IDV and risk-scoring engine.
- Creating a role-gated digital pipeline where every application and claim is tracked in real time.
- Automatically generating policy documents and invoices on payment confirmation using OpenPDF.
- Integrating Razorpay for premium collection and RazorpayX for direct bank payouts.
- Using n8n webhooks to trigger transactional email notifications at every lifecycle event.
- Providing role-specific dashboards for customers, underwriters, claims officers, and admins.

1.4 Scope

VehicleSure covers the following functional domains:

Domain	Scope Included
Customer Onboarding	Self-registration, login, profile management

Quote Engine	IDV calculation, risk scoring, tiered premium options
Policy Application	Application submission, document upload, status tracking
Underwriting Workflow	Assignment, document review, approve/reject
Premium Payment	Razorpay checkout, HMAC verification, policy activation
Document Generation	PDF policy document and invoice auto-generation
Claims Management	Claim filing, officer assignment, payout calculation
Claim Payout	RazorpayX bank transfer, ClaimsPayment record
Administration	Staff lifecycle, plan CRUD, workload balancing, reports
Notifications	In-app notifications, email via n8n webhooks

2. User Roles & Personas

2.1 Customer (ROLE_CUSTOMER)

The policyholder. Self-registered via the public registration endpoint. This is the primary end-user of the platform.

Capabilities:

- Register, login, and change password.
- View active policy plans and request premium quotes.
- Submit vehicle insurance applications with RC and invoice document uploads.
- Pay policy premiums via Razorpay checkout.
- Download generated policy PDF and invoice PDF after payment.
- File claims against active policies with up to 3 supporting documents.
- View all applications, policies, and claims with real-time status tracking.
- Receive in-app and email notifications on every lifecycle event.

Restrictions:

- Cannot view or modify other customers' data.
- Cannot access admin, underwriter, or claims officer dashboards.
- Cannot file a claim on a non-ACTIVE policy.
- Cannot file duplicate applications for the same vehicle registration number.

2.2 Underwriter (ROLE_UNDERWRITER)

Internal staff member created by Admin. Reviews and decides on vehicle insurance applications.

Capabilities:

- Login via Admin-created credentials.
- View dashboard showing only ASSIGNED applications (not all applications).
- Open and review uploaded RC and invoice PDF documents.
- Approve an application (triggers vehicle and policy record creation in PENDING_PAYMENT status).
- Reject an application with a mandatory rejection reason.

Restrictions:

- Cannot view applications not assigned to them.
- Cannot access claim management or admin functions.
- Account must be activated by Admin to log in.

2.3 Claims Officer (ROLE_CLAIMS_OFFICER)

Internal staff member created by Admin. Assesses and decides on insurance claims.

Capabilities:

- Login via Admin-created credentials.
- View dashboard showing only ASSIGNED claims.
- Use the payout calculator tool: input repair bill amount and vehicle year to compute eligible payout.
- View depreciation breakdown (IDV depreciation + parts depreciation) before deciding.
- Approve a claim with a calculated payout amount.
- Reject a claim with a mandatory rejection reason.

Restrictions:

- Cannot view claims not assigned to them.
- Cannot process the actual bank payout (that is Admin's responsibility).
- Cannot access policy application or admin functions.

2.4 Administrator (ROLE_ADMIN)

The super-user. Manually seeded in the database. Manages the entire operational platform.

Capabilities:

- Create, activate, and deactivate Underwriter and Claims Officer accounts.
- View all applications and assign them to underwriters (with workload view).
- View all claims and assign them to claims officers (with workload view).
- Perform full CRUD on insurance policy plans (products).
- Override application status via direct status update endpoint.
- Authorize and process final claim payouts via RazorpayX.
- Export received premium payments and claim payout history as Excel (.xlsx) files.
- View global dashboards for all applications and claims.

2.5 Guest (Unauthenticated)

Any unauthenticated visitor. Access is strictly limited to:

- POST /api/auth/register — create a new customer account.
- POST /api/auth/login — authenticate and receive a JWT token.

All other endpoints return HTTP 401 Unauthorized for unauthenticated requests.

3. Functional Requirements

3.1 Authentication & Authorization (FR-AUTH)

FR-ID	Requirement Description
FR-AUTH-01	Users shall register with: username, email (unique), password (min 8 chars), fullName, and phoneNumber (unique). Default role assigned: CUSTOMER. Returns HTTP 201 on success.
FR-AUTH-02	Users shall authenticate via email + password. On success, the system returns a signed JWT token (HMAC-SHA256). Returns HTTP 200 with token in body.
FR-AUTH-03	All non-public endpoints shall require a valid Authorization: Bearer <token> header. Requests with missing or invalid tokens return HTTP 401 Unauthorized.
FR-AUTH-04	Authenticated users shall change their password by providing the current password for verification before setting a new one. Returns HTTP 200 on success.
FR-AUTH-05	The system shall enforce method-level RBAC via @PreAuthorize("hasRole('ROLE_X')"). Access by an unauthorized role returns HTTP 403 Forbidden.
FR-AUTH-06	Duplicate email or phone number registration shall return HTTP 409 Conflict with a descriptive error message.
FR-AUTH-07	Inactive staff accounts (isActive = false) shall be blocked from login with HTTP 401.

3.2 Quote Generation & Policy Management (FR-POLICY)

FR-ID	Requirement Description
FR-POLICY-01	Authenticated customers shall request a premium quote by providing: vehicleType, make, model, year, fuelType, exShowroomPrice, distanceDriven, and accidentsInPast.
FR-POLICY-02	The system shall calculate IDV using: IDV = exShowroomPrice × (1 − min(vehicleAgeYears × 0.10, 0.50)). Depreciation is capped at 50% regardless of vehicle age.
FR-POLICY-03	The system shall compute a risk score: riskScore = (vehicleAge > 5 ? 1 : 0) + accidentsInPast + (distanceDriven > 100,000 ? 1 : 0). Score 0–1 = LOW (×1.0), Score 2 = MEDIUM (×1.2), Score 3+ = HIGH (×1.5).
FR-POLICY-04	The system shall return three premium options: 1-year (base annual), 2-year (×1.9 of annual), 3-year (×2.8 of annual) for each matching active plan.
FR-POLICY-05	A policy plan shall appear in quote results only if its applicableVehicleType matches the requested vehicleType AND its isActive flag is true.
FR-POLICY-06	Customer shall submit a policy application with: vehicleOwnerName, registrationNumber (regex: ^[A-Z]{2}[0-9]{2}[A-Z]{2}[0-9]{4}$), chassisNumber (17-char VIN), plus rcDocument and invoiceDocument as multipart files.
FR-POLICY-07	The system shall reject any application where an active application already exists for the same registrationNumber (HTTP 409 Conflict).
FR-POLICY-08	Application status flow: UNDER_REVIEW → ASSIGNED → APPROVED → PAID, or UNDER_REVIEW → ASSIGNED → REJECTED.

FR-POLICY-09	On underwriter approval, the system shall automatically create a Vehicle record and a Policy record with status PENDING_PAYMENT.
FR-POLICY-10	On Razorpay payment signature verification, the system shall call paymentService.finalizePayment() to: record the payment (PAID), set policy to ACTIVE, set startDate and endDate, generate PDF invoice and PDF policy document, and save their file paths.
FR-POLICY-11	Customers shall download their generated invoice PDF and policy document PDF at any time after payment confirmation.

3.3 Claims Management (FR-CLAIMS)

FR-ID	Requirement Description
FR-CLAIMS-01	Customers shall only file a claim against a policy with status ACTIVE. Attempting to claim on non-ACTIVE policies returns HTTP 400.
FR-CLAIMS-02	Supported claim types: DAMAGE (own vehicle damage), THEFT (vehicle stolen), THIRD_PARTY (liability to another party).
FR-CLAIMS-03	Claim submission shall accept up to 3 supporting evidence documents (photos, police report, etc.) as optional multipart file uploads.
FR-CLAIMS-04	Claim status flow: SUBMITTED → ASSIGNED → APPROVED → SETTLED, or SUBMITTED → ASSIGNED → REJECTED.
FR-CLAIMS-05	The payout calculator tool shall accept billAmount (repair estimate) and yearOfManufacture. It shall compute and return: IDV depreciation, parts depreciation, depreciated bill amount, total loss check, deductible deduction, and final payout.
FR-CLAIMS-06	IDV Depreciation rates by vehicle age: 0yr=5%, 1yr=15%, 2yr=20%, 3yr=30%, 4yr=40%, 5yr=50%, 5+yr=70%.
FR-CLAIMS-07	Parts Depreciation rates by vehicle age: 0yr=25%, 1yr=30%, 2yr=32%, 3yr=35%, 4yr=38%, 5yr=40%, 5+yr=50%.
FR-CLAIMS-08	Total Loss Rule: if repairCost > 75% of IDV, the payout = full IDV value and the associated policy status is set to INACTIVE.
FR-CLAIMS-09	For ZERO_DEPRECIATION plan types, the full bill amount (no depreciation applied) shall be used for DAMAGE claim calculations.
FR-CLAIMS-10	Final payout formula: finalPayout = max(0, depreciatedBillOrIDV − deductibleAmount). Minimum payout is Rs. 0; negative payouts are not permitted.
FR-CLAIMS-11	On Admin payout execution, the system shall call RazorpayX API with the customer's stored bankAccountNumber, ifscCode, and accountHolderName to execute the bank transfer.
FR-CLAIMS-12	A ClaimsPayment record shall be created recording: amountPaid, transactionReference, paymentStatus (SUCCESS or MOCK_SUCCESS), bankAccountNumber, ifscCode, accountHolderName, and adminId.

3.4 Administration (FR-ADMIN)

FR-ID	Requirement Description
FR-ADMIN-01	Admin shall create Underwriter and Claims Officer accounts with auto-generated or custom passwords. A welcome email with credentials is sent via n8n webhook.
FR-ADMIN-02	Admin shall deactivate (soft-delete) and reactivate staff accounts. Deactivated accounts cannot log in.
FR-ADMIN-03	Admin shall view all policy applications globally (not filtered by assignment).
FR-ADMIN-04	Admin shall view underwriter workload (count of ASSIGNED applications per underwriter) before assigning to balance load.
FR-ADMIN-05	Admin shall assign applications to underwriters via PUT /api/admin/applications/{id}/assign/{underwriterId}.
FR-ADMIN-06	Admin shall override application status directly via PUT /api/admin/applications/{id}/status.
FR-ADMIN-07	Admin shall view all claims globally and view claims officer workload (count of ASSIGNED claims per officer).
FR-ADMIN-08	Admin shall assign claims to claims officers via PUT /api/admin/claims/{id}/assign/{officerId}.
FR-ADMIN-09	Admin shall authorize and execute final claim payouts via POST /api/admin/claims/{id}/pay. This triggers the RazorpayX transfer and updates claim status to SETTLED.
FR-ADMIN-10	Admin shall perform full CRUD on PolicyPlans: create, read, deactivate (soft-delete), and reactivate. Only active plans are visible to customers.
FR-ADMIN-11	Admin shall export received premium payment history as Excel (.xlsx) via GET /api/admin/export/received-payments.
FR-ADMIN-12	Admin shall export claim payout history as Excel (.xlsx) via GET /api/admin/export/payout-history.

3.5 Notifications (FR-NOTIF)

FR-ID	Requirement Description
FR-NOTIF-01	The system shall generate in-app Notification records for all relevant roles on lifecycle events: application assigned, approved, rejected; payment confirmed; claim assigned, approved, rejected; claim settled.
FR-NOTIF-02	The backend shall HTTP POST to the configured N8N_WEBHOOK_URL for email delivery on: new staff onboarding (credentials), application status changes, claim status changes, and settlement confirmation.
FR-NOTIF-03	n8n webhook payload format: { "email": "...", "subject": "...", "body": "..." }.
FR-NOTIF-04	Email notification failures shall be handled gracefully (try-catch). A failed email shall NOT prevent the core business operation from completing. Failures are logged.
FR-NOTIF-05	Authenticated users shall retrieve their notification list via GET /api/notifications. Each notification includes: id, message, type, isRead, createdAt.

4. Non-Functional Requirements

4.1 Performance

Metric	Target	Rationale
Quote generation response time	**< 500 ms**	Real-time user experience
PDF policy/invoice generation	**< 3 seconds**	Post-payment confirmation
Standard API response time	**< 1 second**	Normal load conditions
Excel export generation	**< 5 seconds**	Large dataset handling
System availability (production)	**99.9% uptime**	Business continuity

4.2 Security

- **Password Hashing**: BCrypt with Spring Security BCryptPasswordEncoder (default strength factor 10). No plaintext passwords stored or returned in any API response.
- **Token Signing**: JWT tokens signed with HMAC-SHA256 using a configured secret (JWT_SECRET env var). Tokens include userId, username, role, iat, exp claims.
- **Endpoint Security**: Spring Security 6 with @EnableMethodSecurity. Every endpoint protected via @PreAuthorize("hasRole('ROLE_X')")-level RBAC.
- **Payment Signature Verification**: Razorpay webhook payloads verified via HMAC-SHA256: HMAC(orderId + "|" + paymentId, keySecret). Mismatch returns HTTP 400.
- **Input Validation**: Jakarta Bean Validation on all request DTOs. @Valid annotation on all @RequestBody parameters. Violations return HTTP 400 with field-level messages.
- **CSRF**: Disabled — acceptable for stateless JWT APIs with no cookie-based sessions.
- **Sensitive Config**: All secrets (DB credentials, Razorpay keys, JWT secret) supplied via .env environment variables, never hardcoded. .env is gitignored.

4.3 Scalability

- The notification service is fully decoupled from core services; it can be upgraded to a message queue (RabbitMQ, Kafka) without core changes.
- The Spring Boot backend is designed as a stateless REST service to support horizontal scaling behind a load balancer.
- Hibernate ORM provides database-agnostic access; migrating to PostgreSQL requires only a JDBC driver change.
- Angular SPA is a deployable static build artifact — can be served by any CDN or web server.

4.4 Data Integrity

- All service methods performing multi-step database operations are annotated @Transactional for atomicity.

- Failed RazorpayX payout API calls throw exceptions that prevent claim status from being updated to SETTLED.
- Duplicate active applications for the same vehicle registration number are rejected at the service layer (not just DB constraint).
- Policy numbers follow a deterministic format (e.g., POL-YYYYMMDD-NNNN) for auditability.

4.5 Usability

- Angular frontend is mobile-responsive using Tailwind CSS utility classes.
- All forms provide real-time client-side validation using Angular Reactive Forms.
- Global HTTP Interceptor catches all API errors and displays user-friendly in-app messages.
- Role-based routing and Angular AuthGuard prevent unauthorized navigation.
- API documentation auto-generated via Springdoc OpenAPI and accessible at /swagger-ui.html.

4.6 Maintainability

- Backend follows Controller → Service → Repository layered architecture for separation of concerns.
- All DTOs (Request/Response) are separate from JPA entities to decouple API contract from DB schema.
- Environment-specific config externalized via .env and application.properties.
- Springdoc OpenAPI provides self-documenting API specification for onboarding new developers.

5. Data Models & Entities

5.1 MyUser (Table: users)

Field	Type	Constraints	Description
`id`	Long	PK, AUTO_INCREMENT	System-generated user identifier
`username`	String	NOT NULL, max 100	Display name / login handle
`password`	String	NOT NULL, max 255	BCrypt-hashed password
`email`	String	UNIQUE, max 150	Primary login credential; must be unique
`fullName`	String	nullable	Full legal name of the user
`phoneNumber`	String	NOT NULL, UNIQUE, max 15	Mobile number; must be globally unique
`role`	UserRole (enum)	NOT NULL	ADMIN, UNDERWRITER, CLAIMS_OFFICER, CUSTOMER
`isActive`	boolean	DEFAULT true	Soft-delete flag; false = account blocked
`createdAt`	LocalDateTime	NOT NULL, non-updatable	Account creation timestamp

Relationships:

- OneToMany → Vehicle (customer owns vehicles)
- OneToMany → Policy (customer as policyholder; underwriter as approver)
- OneToMany → VehicleApplication (customer submitted; underwriter assigned)
- OneToMany → Claims (customer filed; claims officer handled)

5.2 Vehicle (Table: vehicles)

Field	Type	Constraints	Description
`vehicleId`	Integer	PK, AUTO_INCREMENT	System-generated vehicle ID
`registrationNumber`	String	NOT NULL, UNIQUE	Vehicle registration plate number
`make`	String	nullable	Manufacturer (e.g., Maruti, Hyundai)
`model`	String	nullable	Model name (e.g., Swift, i20)
`year`	Integer	nullable	Manufacturing year
`fuelType`	String	nullable	Petrol, Diesel, CNG, Electric

`vehicleType`	VehicleType (enum)	NOT NULL	CAR, EV_CAR, TWO_WHEELER, EV_TWO_WHEELER, AUTO, EV_AUTO, HEAVY_VEHICLE
`transmissionType`	TransmissionType	NOT NULL	MANUAL, AUTOMATIC
`accidentsInPast`	Integer	NOT NULL	Number of prior accidents
`riskLevel`	RiskLevel (enum)	NOT NULL	LOW, MEDIUM, HIGH
`customer_id`	Long	FK → users.id, NOT NULL	Vehicle owner (customer)
`vehicle_application_id`	Integer	FK → vehicle_applications.id	Linked application record

5.3 VehicleApplication (Table: vehicle_applications)

Field	Type	Constraints	Description
`vehicleApplicationId`	Integer	PK, AUTO_INCREMENT	System-generated application ID
`vehicleOwnerName`	String	nullable	Name as on RC document
`registrationNumber`	String	nullable	Vehicle registration plate
`chassisNumber`	String	nullable	17-character VIN
`distanceDriven`	Long	nullable	Total km driven
`vehicleType`	VehicleType (enum)	NOT NULL	Vehicle category
`transmissionType`	TransmissionType	NOT NULL	Gearbox type
`accidentsInPast`	Integer	NOT NULL	Accident history count
`riskLevel`	RiskLevel (enum)	NOT NULL	Computed risk classification
`exShowroomPrice`	BigDecimal	nullable	Original showroom price (INR)
`idv`	BigDecimal	nullable	Insured Declared Value
`calculatedPremium`	BigDecimal	nullable	Premium from quote engine
`tenureYears`	Integer	nullable	Policy duration in years
`status`	VehicleApplicationStatus	DEFAULT UNDER_REVIEW	UNDER_REVIEW, ASSIGNED, APPROVED, REJECTED, PAID
`rejectionReason`	String	nullable	Populated on underwriter rejection
`createdAt`	LocalDateTime	DEFAULT now()	Application submission timestamp
`customer_id`	Long	FK → users.id, NOT NULL	Applicant (customer)
`assigned_underwriter_id`	Long	FK → users.id, nullable	Assigned underwriter
`plan_id`	Integer	FK → policy_plans.id, nullable	Selected insurance plan

5.4 PolicyPlan (Table: policy_plans)

Field	Type	Constraints	Description
`planId`	Integer	PK, AUTO_INCREMENT	System plan ID
`planName`	String	nullable	Display name (e.g., Gold Shield)
`policyType`	String	nullable	COMPREHENSIVE, THIRD_PARTY
`basePremium`	BigDecimal	nullable	Base premium before risk multiplier
`maxCoverageAmount`	BigDecimal	nullable	Maximum coverage ceiling (INR)
`deductibleAmount`	BigDecimal	nullable	Customer out-of-pocket deductible
`coversThirdParty`	boolean	—	Third-party liability coverage flag
`coversOwnDamage`	boolean	—	Own damage coverage flag
`coversTheft`	boolean	—	Theft coverage flag
`zeroDepreciationAvailable`	boolean	—	Zero-dep add-on availability flag
`engineProtectionAvailable`	boolean	—	Engine protection add-on flag
`roadsideAssistanceAvailable`	boolean	—	Roadside assistance add-on flag
`applicableVehicleType`	VehicleType (enum)	NOT NULL	Vehicle category this plan covers
`isActive`	boolean	DEFAULT true	Visibility to customers in quote engine
`createdAt`	LocalDateTime	NOT NULL	Plan creation timestamp

5.5 Policy (Table: policies)

Field	Type	Constraints	Description
`policyId`	Integer	PK, AUTO_INCREMENT	System policy ID
`policyNumber`	String	NOT NULL, UNIQUE	e.g., POL-20260318-0001
`status`	PolicyStatus (enum)	—	DRAFT, PENDING_PAYMENT, ACTIVE, INACTIVE, EXPIRED, CANCELLED
`startDate`	LocalDate	nullable	Coverage start date
`endDate`	LocalDate	nullable	Coverage end date
`premiumAmount`	BigDecimal	nullable	Final premium paid by customer
`customer_id`	Long	FK → users.id, NOT NULL	Policyholder
`underwriter_id`	Long	FK → users.id, nullable	Approving underwriter
`vehicle_id`	Integer	FK → vehicles.id, NOT NULL	Insured vehicle
`plan_id`	Integer	FK → policy_plans.id, NOT NULL	Linked insurance plan
`invoicePath`	TEXT	nullable	Server filesystem path to invoice PDF
`policyDocumentPath`	TEXT	nullable	Server filesystem path to policy PDF

5.6 Payment (Table: payments)

Field	Type	Constraints	Description
`paymentId`	Integer	PK, AUTO_INCREMENT	System payment ID
`amount`	BigDecimal	nullable	Amount paid in INR
`paymentDate`	LocalDateTime	nullable	Timestamp of payment confirmation
`status`	PaymentStatus (enum)	—	PENDING, PAID, FAILED
`transactionReference`	String	nullable	Razorpay payment ID (razorpay_payment_id)
`policy_id`	Integer	FK → policies.id, NOT NULL	Associated policy record

5.7 Claims (Table: claims)

Field	Type	Constraints	Description
`claimId`	Integer	PK, AUTO_INCREMENT	System claim ID
`claimNumber`	String	UNIQUE	e.g., CLM-20260318-0001
`claimType`	ClaimType (enum)	—	THEFT, DAMAGE, THIRD_PARTY
`status`	ClaimStatus (enum)	max 50	SUBMITTED, ASSIGNED, APPROVED, REJECTED, SETTLED
`approvedAmount`	BigDecimal	nullable	Payout amount approved by claims officer
`rejectionReason`	String	max 1000, nullable	Reason for rejection
`policy_id`	Integer	FK → policies.id, NOT NULL	Associated policy
`customer_id`	Long	FK → users.id, NOT NULL	Claimant (customer)
`claims_officer_id`	Long	FK → users.id, nullable	Assigned claims officer
`bankAccountNumber`	String	nullable	Destination account for payout
`ifscCode`	String	nullable	Bank branch IFSC code
`accountHolderName`	String	nullable	Beneficiary name on bank account

5.8 ClaimDocument (Table: claim_documents)

Field	Type	Constraints	Description
`documentId`	Integer	PK, AUTO_INCREMENT	System document ID
`document1`	String	nullable	File path to evidence document 1
`document2`	String	nullable	File path to evidence document 2
`document3`	String	nullable	File path to evidence document 3
`claim_id`	Integer	FK → claims.id, NOT NULL	Parent claim record

5.9 ClaimsPayment (Table: claims_payment)

Field	Type	Constraints	Description
`paymentId`	Integer	PK, AUTO_INCREMENT	System payment ID
`claim_id`	Integer	FK → claims.id, NOT NULL	OneToOne with claim
`admin_id`	Long	FK → users.id, NOT NULL	Admin who triggered the payout
`amountPaid`	BigDecimal	NOT NULL	Actual payout amount disbursed
`paymentDate`	LocalDateTime	NOT NULL, @PrePersist	Payout execution timestamp
`paymentStatus`	String	NOT NULL	SUCCESS or MOCK_SUCCESS
`transactionReference`	String	NOT NULL	RazorpayX fund account ID or mock ref
`bankAccountNumber`	String	nullable	Destination bank account used
`ifscCode`	String	nullable	IFSC code used for transfer
`accountHolderName`	String	nullable	Beneficiary name used

5.10 Notification (Table: notifications)

Field	Type	Constraints	Description
`id`	Long	PK, AUTO_INCREMENT	System notification ID
`user_id`	Long	FK → users.id, NOT NULL	Target user for this notification
`message`	String	NOT NULL, max 500	Notification text content

`type`	String	NOT NULL	POLICY_UPDATE, CLAIM_UPDATE, PAYMENT_SUCCESS, NEW_ASSIGNMENT
`isRead`	boolean	NOT NULL, DEFAULT false	Whether the user has read this notification
`createdAt`	LocalDateTime	NOT NULL, DEFAULT now()	Notification creation timestamp

5.11 VehicleDocument (Table: vehicle_documents)

Field	Type	Constraints	Description
`documentId`	Integer	PK, AUTO_INCREMENT	System document ID
`documentType`	String	nullable	RC_BOOK or INVOICE
`filePath`	String	nullable	Server-side file system path to document
`vehicle_application_id`	Integer	FK → vehicle_applications.id, NOT NULL	Parent application

5.12 Entity Relationship Summary

From Entity	Cardinality	To Entity	Description
MyUser	1 → N	Vehicle	Customer owns vehicles
MyUser	1 → N	Policy	Customer holds / UW approves
MyUser	1 → N	VehicleApplication	Customer submits / UW reviews
MyUser	1 → N	Claims	Customer files / Officer handles
VehicleApplication	1 → N	VehicleDocument	Application has RC+invoice docs
Vehicle	1 → N	Policy	Vehicle is insured by policies
PolicyPlan	1 → N	Policy	Plan is the product template
Policy	1 → N	Payment	Policy has premium payments
Policy	1 → N	Claims	Policy can have multiple claims

Claims	1 → N	ClaimDocument	Claim has evidence documents
Claims	1 → 1	ClaimsPayment	Settled claim has one payout record

6. API Reference

6.1 Authentication APIs (/api/auth)

Method	Endpoint	Role	Request / Response
POST	/api/auth/register	Public	Body: RegisterReqDTO {username, password, email, phoneNumber} → RegisterResDTO {id, username, email, role} | HTTP 201
POST	/api/auth/login	Public	Body: JwtRequest {email, password} → JwtResponse {token} | HTTP 200
PUT	/api/auth/change-password	Authenticated	Body: ChangePasswordDTO {currentPassword, newPassword} → String message | HTTP 200

> *NOTE: Auth errors: 400 Bad Request (validation), 401 Unauthorized (bad credentials), 409 Conflict (duplicate email/phone).*

6.2 Customer APIs (/api/customer) — ROLE_CUSTOMER

Method	Endpoint	Request	Response
POST	/api/customer/buy-policy	multipart: application (JSON), rcDocument (file), invoiceDocument (file)	String — 'Application submitted' | HTTP 200
GET	/api/customer/applications	—	List<ReadVehicleApplicationDTO> (filtered by customerId)
GET	/api/customer/policies	—	List<ReadPolicyDTO> (filtered by customerId)
GET	/api/customer/policy-plans	Query: ?type=CAR (optional)	List<ReadPolicyPlanDTO> — active plans only
POST	/api/customer/quote	Body: QuoteRequestDTO {vehicleType, make, model, year, fuelType, exShowroomPrice, distanceDriven, accidentsInPast}	List<QuoteResponseDTO> {planId, planName, idv, premium1yr, premium2yr, premium3yr}
POST	/api/customer/submit-claim	multipart: claim (JSON), doc1, doc2, doc3 (optional files)	ReadClaimDTO | HTTP 200
GET	/api/customer/my-claims	—	List<ReadClaimDTO> (filtered by customerId)

6.3 Payment APIs (/api/payments) — ROLE_CUSTOMER

Method	Endpoint	Request	Response
POST	/api/payments/create-order	Body: PaymentRequestDTO {policyId, amount}	RazorpayOrderResponseDTO {orderId, amount, currency, key}
POST	/api/payments/verify	Body: {razorpayPaymentId, razorpayOrderId, razorpaySignature, policyId}	String — 'Payment verified' | HTTP 400 on signature mismatch
POST	/api/payments/process	Body: PaymentRequestDTO	String — records payment & triggers PDF generation
GET	/api/payments/download/invoice/{policyId}	Path: policyId	PDF binary (application/pdf)
GET	/api/payments/download/policy/{policyId}	Path: policyId	PDF binary (application/pdf)

6.4 Underwriter APIs (/api/underwriter) — ROLE_UNDERWRITER

Method	Endpoint	Request	Response
GET	/api/underwriter/applications	—	List<ReadVehicleApplicationDTO> — only assigned to authenticated UW
POST	/api/underwriter/applications/{id}/approve	Path: id	HTTP 200 — creates Policy (PENDING_PAYMENT) + Vehicle records
POST	/api/underwriter/applications/{id}/reject	Body: {reason: String}	HTTP 200 — stores rejection reason, notifies customer

6.5 Claims Officer APIs (/api/claims-officer) — ROLE_CLAIMS_OFFICER

Method	Endpoint	Request	Response
GET	/api/claims-officer/claims	—	List<ReadClaimDTO> — only assigned to authenticated officer
POST	/api/claims-officer/claims/{id}/approve	Body: ApproveClaimRequest {approvedAmount}	HTTP 200 — sets status to APPROVED

POST	/api/claims-officer/claims/{id}/payout	Body: {repairEstimate, vehicleAge}	BigDecimal — preview payout (does NOT approve)
POST	/api/claims-officer/claims/{id}/reject	Body: RejectClaimRequest {reason}	HTTP 200 — sets status to REJECTED

6.6 Admin APIs (/api/admin) — ROLE_ADMIN

Staff Management:

Method	Endpoint	Request	Response
POST	/api/admin/staff	Body: CreateStaffDTO	Map<String,String> — success message + credentials
DELETE	/api/admin/staff/{id}	Path: id	ReadStaffDTO — deactivated staff
PUT	/api/admin/staff/{id}/activate	Path: id	ReadStaffDTO — reactivated staff
GET	/api/admin/staff	—	List<ReadStaffDTO>
GET	/api/admin/staff/{id}	Path: id	ReadStaffDTO

Policy Plan Management:

Method	Endpoint	Request	Response
POST	/api/admin/policy-plans	Body: CreatePolicyPlanDTO	PolicyPlan (created)
GET	/api/admin/policy-plans	—	List<ReadPolicyPlanDTO>
GET	/api/admin/policy-plans/{id}	Path: id	ReadPolicyPlanDTO
DELETE	/api/admin/policy-plans/{id}	Path: id	ReadPolicyPlanDTO (deactivated)
PUT	/api/admin/policy-plans/{id}/activate	Path: id	ReadPolicyPlanDTO (reactivated)

Application Management:

Method	Endpoint	Request	Response
GET	/api/admin/applications	—	List<ReadVehicleApplicationDTO> — all applications
PUT	/api/admin/applications/{id}/status	Body: UpdateApplicationStatusDTO	ReadVehicleApplicationDTO
PUT	/api/admin/applications/{id}/assign/{underwriterId}	Path params	ReadVehicleApplicationDTO
GET	/api/admin/underwriters/workload	—	List<UnderwriterWorkloadDTO> {underwriterId, name, assignedCount}

Claims & Payouts:

Method	Endpoint	Request	Response
GET	/api/admin/claims	—	List<ReadClaimDTO> — all claims
GET	/api/admin/claims-officers/workload	—	List<ClaimsOfficerWorkloadDTO> {officerId, name, assignedCount}
PUT	/api/admin/claims/{id}/assign/{officerId}	Path params	ReadClaimDTO
GET	/api/admin/claims/pending-payments	—	List<ReadClaimDTO> — APPROVED, not yet SETTLED
POST	/api/admin/claims/{id}/pay	Path: id	Map<String,String> — triggers RazorpayX payout
GET	/api/admin/received-payments	—	List<ReadPaymentDTO>
GET	/api/admin/claims/payouts	—	List<ReadClaimsPaymentDTO>
GET	/api/admin/export/received-payments	—	XLSX binary download
GET	/api/admin/export/payout-history	—	XLSX binary download

6.7 Notification API (/api/notifications)

Method	Endpoint	Role	Response
GET	/api/notifications	Authenticated	List<NotificationDTO> {id, message, type, isRead, createdAt}

7. Workflow & State Machines

7.1 VehicleApplication Status Flow

```
[UNDER_REVIEW]  ──(Admin assigns)──────────────▶  [ASSIGNED]
[ASSIGNED]      ──(Underwriter approves)──────▶  [APPROVED]
[ASSIGNED]      ──(Underwriter rejects)───────▶  [REJECTED]
[APPROVED]      ──(Customer pays via Razorpay)──▶  [PAID]
[ASSIGNED]      ──(Admin override)────────────▶  [any status]
```

Transition	Trigger	Automated Side Effects
UNDER_REVIEW → ASSIGNED	Admin assigns underwriter	assignedUnderwriter set; in-app + email notification to underwriter
ASSIGNED → APPROVED	Underwriter approves	Vehicle record created; Policy created (PENDING_PAYMENT); customer notified
ASSIGNED → REJECTED	Underwriter rejects	rejectionReason stored; customer notified via in-app + email
APPROVED → PAID	Razorpay payment verified	Policy → ACTIVE; PDF invoice + policy doc generated; payment recorded; customer notified

7.2 Policy Status Flow

```
[DRAFT]            ──(App approval)──────────────▶  [PENDING_PAYMENT]
[PENDING_PAYMENT]  ──(Payment verified)──────────▶  [ACTIVE]
[ACTIVE]           ──(Admin cancels)─────────────▶  [CANCELLED]
[ACTIVE]           ──(End date passed — future)──▶  [EXPIRED]
[ACTIVE]           ──(Total loss claim settled)──▶  [INACTIVE]
```

Transition	Trigger	Automated Side Effects
DRAFT → PENDING_PAYMENT	Underwriter approves application	Policy number assigned (POL-YYYYMMDD-NNNN); customer can now pay
PENDING_PAYMENT → ACTIVE	paymentService.finalizePayment()	Payment recorded (PAID); PDF invoice generated; PDF policy document generated; startDate/endDate set; customer notified
ACTIVE → INACTIVE	Total loss claim settled	Policy set INACTIVE; further claims blocked

7.3 Claim Status Flow

```
[SUBMITTED]  ──(Admin assigns)──────────────────▶  [ASSIGNED]
[ASSIGNED]   ──(Officer approves + amount)──────▶  [APPROVED]
[ASSIGNED]   ──(Officer rejects + reason)───────▶  [REJECTED]
[APPROVED]   ──(Admin executes payout)──────────▶  [SETTLED]
```

Transition	Trigger	Automated Side Effects
SUBMITTED → ASSIGNED	Admin assigns officer	claimsOfficer set; in-app + email notification to officer
ASSIGNED → APPROVED	Officer approves with amount	approvedAmount stored; customer notified
ASSIGNED → REJECTED	Officer rejects with reason	rejectionReason stored; customer notified
APPROVED → SETTLED	Admin executes payout	RazorpayX transfer executed; ClaimsPayment record created; claim status = SETTLED; customer notified

7.4 Payment Status Flow

```
[PENDING]  ──(Razorpay signature verified)──────▶  [PAID]
[PENDING]  ──(Razorpay failure / timeout)───────▶  [FAILED]
```

> *NOTE: A FAILED payment does not block the customer from retrying. The policy remains in PENDING_PAYMENT status until a successful payment is recorded.*

8. Third-Party Integrations

8.1 Razorpay — Premium Payment Gateway

Purpose	Collect policy premium payments from customers
Auth Method	RAZORPAY_KEY_ID + RAZORPAY_KEY_SECRET (from .env)
Test Mode	Yes — rzp_test_* key used in development
Amount Unit	Paise (INR × 100). e.g., Rs. 5000 = 500000 paise
Failure Handling	Invalid HMAC signature → HTTP 400; policy stays in PENDING_PAYMENT; customer can retry

Integration Flow:

1. Backend creates Razorpay order via POST /v1/orders with amount, currency=INR, receipt, notes.
2. Frontend renders Razorpay Checkout widget with {orderId, amount, currency, key}.
3. Customer completes payment in the Razorpay UI modal.
4. Razorpay returns {razorpay_payment_id, razorpay_order_id, razorpay_signature} to frontend.
5. Frontend submits these to POST /api/payments/verify.
6. Backend verifies: HMAC-SHA256(orderId + '|' + paymentId, keySecret) must match signature.
7. On match → finalizePayment() called → policy activated, PDFs generated, payment recorded.

8.2 RazorpayX — Claim Payout Gateway

Purpose	Disburse approved claim amounts directly to customer bank accounts
Auth Method	Same RAZORPAY_KEY_ID + RAZORPAY_KEY_SECRET
Mock Mode	If API unavailable, system creates ClaimsPayment with paymentStatus=MOCK_SUCCESS
Failure Handling	On API exception → fallback to mock; exception is logged; ClaimsPayment created
Data Sent	accountNumber, ifscCode, accountHolderName, amount (INR), currency, purpose

Integration Flow:

8. Admin triggers POST /api/admin/claims/{id}/pay.
9. Backend calls RazorpayX to: (a) create a contact, (b) create a fund account, (c) initiate a payout.
10. Funds transferred directly to customer bank account via NEFT/RTGS.
11. ClaimsPayment record created with transactionReference and paymentStatus.
12. Claim status updated to SETTLED; customer receives in-app and email notification.

8.3 n8n — Email Notification Webhook

Purpose	Send transactional email notifications on key lifecycle events
Auth Method	Webhook URL as secret (security by obscurity — MVP limitation)
Failure Handling	Non-blocking; in-app notification is the primary channel; failures are logged silently
Payload Format	{ "email": "user@example.com", "subject": "...", "body": "..." }
Trigger Events	Staff onboarding, application assigned/approved/rejected, payment confirmed, claim assigned/approved/rejected/settled

8.4 Apache POI — Excel Report Generation

- **Purpose**: Generate .xlsx Excel reports for admin export
- **Reports**: received_payments.xlsx (all premium payments) and payout_history.xlsx (all claim payouts)
- **Trigger**: Admin GET /api/admin/export/* endpoints
- **Output**: ByteArrayInputStream streamed as InputStreamResource with application/vnd.openxmlformats-officedocument.spreadsheetml.sheet

8.5 OpenPDF — Policy & Invoice Document Generation

- **Purpose**: Auto-generate policy document and tax invoice PDFs on payment confirmation
- **Trigger**: paymentService.finalizePayment() on successful Razorpay signature verification
- **Invoice Content**: Policy number, customer name, vehicle details, premium amount, date, tax breakdown
- **Policy Doc Content**: Policy terms, coverage details, vehicle info, validity period, policy number, plan features
- **Storage**: PDF files saved to /uploads/ directory; paths stored in Policy.invoicePath and Policy.policyDocumentPath

9. System Architecture & Tech Stack

9.1 Frontend

Category	Technology / Detail
Framework	Angular 21 — Standalone Component Architecture
Language	TypeScript
State Management	RxJS Observables + Angular Signals
UI Styling	Tailwind CSS — mobile-responsive utility classes
HTTP Client	Angular HttpClient with global HTTP Interceptor for auth + error handling
Auth	JWT stored in localStorage; injected via HTTP Interceptor on every request
Routing	Angular Router with lazy-loaded modules and AuthGuard for role-based access
Build Tool	Angular CLI (ng serve for dev, ng build for prod)
Dev Port	http://localhost:4200

Project Structure:

```
frontend/src/app/
  ├── core/       (guards, interceptors, services, models)
  ├── features/   (customer, admin, underwriter, claims-officer)
  └── shared/     (reusable components)
```

9.2 Backend

Category	Technology / Detail
Language	Java 17
Framework	Spring Boot 3.3.5
Architecture	Layered REST: Controller → Service → Repository
Security	Spring Security 6 + JWT (JJWT library), HMAC-SHA256
ORM	Spring Data JPA (Hibernate)
Validation	Jakarta Bean Validation (@Valid, @NotBlank, @Pattern, @Size, @Email)
Doc Generation	OpenPDF (PDF), Apache POI (Excel)
API Docs	Springdoc OpenAPI 2 → Swagger UI at /swagger-ui.html
Build	Maven (mvnw wrapper)
Package	org.thehartford.willowshield
Dev Port	http://localhost:8080

9.3 Database

Category	Detail
Engine	MySQL 8.x
Database Name	vehiclesure
Schema Management	Auto-managed by Hibernate (spring.jpa.hibernate.ddl-auto)
Connection	JDBC URL configured via DB_URL environment variable
ORM	Hibernate (via Spring Data JPA)

9.4 Authentication Mechanism

- **Token Type**: Stateless JWT (JSON Web Token)
- **Generation**: JwtUtility.generateToken(userId, username, role) — HMAC-SHA256
- **Validation**: JwtFilter (extends OncePerRequestFilter) — intercepts every request
- **Token Claims**: userId (subject), username, role, iat (issued-at), exp (expiration)
- **Client Storage**: localStorage (Angular side)
- **Session Policy**: STATELESS — zero server-side session state

9.5 File Storage

- **Type**: Local filesystem
- **Upload Dir**: /uploads/ (relative to application working directory)
- **Files Stored**: RC documents, vehicle invoices, claim evidence docs, generated policy PDFs, invoice PDFs
- **Access**: Static serving via Spring Boot for /uploads/** path (permitted in SecurityConfig)
- **Limitation**: No cloud storage (S3/GCS) in MVP. Single-server assumption.

9.6 Environment Variables

Variable	Mandatory	Purpose
`DB_URL`	**YES**	MySQL JDBC URL (jdbc:mysql://localhost:3306/vehiclesure)
`DB_USERNAME`	**YES**	MySQL database username
`DB_PASSWORD`	**YES**	MySQL database password
`JWT_SECRET`	**YES**	HMAC-SHA256 secret key for JWT signing
`JWT_EXPIRATION_MS`	**Optional**	JWT expiry in milliseconds (default set in application.properties)

`RAZORPAY_KEY_ID`	**YES**	Razorpay public key (test: rzp_test_*, live: rzp_live_*)
`RAZORPAY_KEY_SECRET`	**YES**	Razorpay secret for HMAC signature verification
`N8N_WEBHOOK_URL`	**Optional**	n8n webhook endpoint for email dispatch; skipped if absent

10. Security Requirements

10.1 Authentication & Token Lifecycle

- JWT signed with HMAC-SHA256 using JWT_SECRET environment variable.
- Tokens include: userId (as subject), username, role, iat, exp claims.
- JwtFilter validates token on every request before reaching the controller layer.
- Invalid or expired tokens return HTTP 401 Unauthorized.
- STATELESS session policy — no server-side sessions; each request is independently authenticated.

10.2 Password Policy

- **Minimum length**: 8 characters (enforced via @Size(min=8) on DTO)
- **Maximum length**: 50 characters
- **Storage**: BCrypt (BCryptPasswordEncoder, default strength 10)
- **Rule**: No plaintext password stored or returned in any API response

10.3 Role Enforcement

- @EnableMethodSecurity on SecurityConfig enables @PreAuthorize annotations.
- Every controller method protected: @PreAuthorize("hasRole('ROLE_ADMIN')") etc.
- Role extracted from JWT claims and set in SecurityContext by JwtFilter.
- Wrong role → HTTP 403 Forbidden.

10.4 Input Validation

- Jakarta Bean Validation annotations on all request DTOs: @NotBlank, @NotNull, @Email, @Pattern, @Min, @Size.
- @Valid annotation on all @RequestBody controller parameters triggers validation.
- Constraint violations return HTTP 400 with field-level error messages via global exception handler.

10.5 CORS Configuration

```
Allowed Origins:   http://localhost:4200
Allowed Methods:   GET, POST, PUT, DELETE, OPTIONS, PATCH
Allowed Headers:   * (all)
Exposed Headers:   Authorization
Allow Credentials: true
```

> *NOTE: CORS origins must be updated to production domain before deployment.*

10.6 Known Security Limitations

#	Limitation	Risk	Planned Mitigation
1	Bank details stored unencrypted	Data breach exposes account numbers	AES-256 at-rest encryption (future scope)
2	No rate limiting on auth endpoints	Brute-force login attacks	Spring Security + Bucket4j (future scope)
3	No audit log table	Cannot trace who changed what/when	AuditLog entity (future scope)
4	No file MIME type validation	Unexpected file types uploaded	MIME check + virus scan (future scope)
5	n8n webhook URL as only security	Webhook URL exposure → email spam	HMAC-signed webhook payloads
6	CORS locked to localhost only	Cannot deploy to prod without change	Dynamic origin whitelist from config

11. Constraints & Known Limitations

#	Constraint	Current Behavior	Ideal Behavior	Workaround
1	Local file storage	Files stored in /uploads/ on local disk	AWS S3/GCS for scalability	Manual backup of uploads folder
2	No policy renewal	Policies expire silently; no reminder	Scheduled job + renewal prompts	Admin monitors manually
3	No payment retry UI	Failed checkout requires restarting flow	Graceful retry with same order ID	Customer resubmits buy-policy
4	Bank details in plaintext	Account number + IFSC as plain VARCHAR	AES-256 encryption at rest	DB access control as compensating control
5	No refresh token	JWT expires; user must re-login	Silent refresh token mechanism	Short expiry × user re-login
6	CORS locked to localhost	Only localhost:4200 allowed	Dynamic origin whitelist from config	Update env before production deploy
7	No file MIME validation	Any file can be uploaded	PDF/JPEG/PNG validation only	Operational trust at MVP stage
8	No audit log	No record of who changed what/when	Audit log entity with actor + action	None currently
9	No rate limiting	Auth endpoints unprotected from brute force	Bucket4j rate limiting	Firewall-level limits
10	RazorpayX mock fallback	API errors fall back to mock payouts	Explicit retry queue + manual override	Admin verifies each payout result
11	No IRDAI compliance fields	No regulatory reference numbers	IRDAI-specific policy schedule format	MVP scope decision

12. Assumptions & Dependencies

12.1 External Dependencies

Dependency	Version	Purpose
Java	17	Backend runtime environment
Spring Boot	3.3.5	Backend application framework
Maven	3.x	Backend build and dependency management
MySQL	8.x	Primary relational database
Node.js	18+	Frontend build runtime (Angular CLI requirement)
Angular CLI	21.x	Frontend toolchain and build system
Razorpay Java SDK	Latest	Premium payment order creation and verification
Apache POI	5.x	Excel (.xlsx) report generation
OpenPDF	1.x	PDF policy and invoice document generation
JJWT	Latest	JWT generation and validation library
n8n	Self-hosted	Email notification automation via webhook

12.2 Infrastructure Assumptions

- Backend and frontend run on the same machine in development (localhost:8080 and localhost:4200).
- MySQL instance accessible at localhost:3306 with database 'vehiclesure' pre-created.
- /uploads/ directory must exist and be writable by the Java process.
- Development OS: Windows. Production OS: Linux (planned).
- Minimum 4GB RAM for combined MySQL + Spring Boot + Angular dev server.

12.3 Data Assumptions

- At least one Admin user must be manually inserted into the users table with role=ADMIN and a BCrypt-hashed password before system use begins.
- At least one PolicyPlan must be created by Admin via API before customers can generate quotes or submit applications.
- Vehicle year is used as-is for depreciation calculation; no validation against VAHAN registry.
- IDV and premium values submitted by the customer (copied from the quote screen) are trusted during application submission.

13. Future Scope

#	Feature	Description	Why Deferred
1	Policy Renewal	Auto-detect expiring policies, send renewal reminders, allow one-click renewal	Requires scheduled jobs + extra payment flow
2	Cloud File Storage (S3)	Replace /uploads/ with AWS S3 or GCS for scalability and disaster recovery	Infrastructure setup complexity
3	Mobile App	React Native or Flutter customer app for buying policies and filing claims	Frontend-only effort; API is ready
4	Refresh Token Mechanism	Silent JWT refresh to avoid forced re-login	Security design requires session store
5	Rate Limiting	Bucket4j-based rate limiting on /api/auth/login to prevent brute-force attacks	Requires middleware setup
6	Audit Log	Dedicated audit trail table recording all entity state changes with actor ID and timestamp	Data model not finalized
7	At-Rest Encryption	AES-256 encryption of bank account numbers and IFSC codes in the database	Key management setup required
8	VAHAN API Integration	Real-time vehicle registration verification against India's national VAHAN database	API access requires IRDAI/Govt approval
9	Analytics Dashboard	Admin KPI charts: applications per day, claims ratio, revenue trend (Grafana/Chart.js)	Requires BI layer
10	Two-Factor Authentication	OTP via SMS/email on login	Third-party OTP provider needed
11	Payment Retry UI	Allow customer to retry payment on same policy order without re-submitting application	Requires order ID persistence across sessions
12	Claim Investigation Status	Additional INVESTIGATING status between ASSIGNED and APPROVED for complex claims	Claims officer role expansion needed
13	Add-On Coverage Purchase	Let customers add zero-depreciation, engine protection, RSA to existing active policies	Policy amendment API needed
14	Multi-Currency Support	Enable policies priced in currencies other than INR	Payment gateway config complexity

14. Glossary

Term / Acronym	Definition
IDV	Insured Declared Value — maximum sum the insurer pays on total loss or theft. Formula: exShowroomPrice × (1 − min(vehicleAge × 10%, 50%))
Premium	Amount paid by the customer to purchase insurance coverage for a specified period
Policy Plan	Configurable insurance product template (e.g., Gold Shield) defining coverage types, base premium, duration, and deductible
Policy	Active insurance contract issued to a customer for a specific vehicle under a specific plan
Underwriter	Internal staff responsible for evaluating vehicle applications and deciding to approve or reject coverage
Claims Officer	Internal staff reviewing filed claims, assessing damage, and computing/approving payouts
VehicleApplication	Customer's formal request to purchase insurance — contains vehicle info, uploaded documents, and selected quote details
Claim	Formal request by a policyholder to receive compensation for a covered loss event
ClaimType	Category of loss: THEFT (vehicle stolen), DAMAGE (own damage), THIRD_PARTY (liability to another party)
RiskLevel	Computed classification (LOW / MEDIUM / HIGH) based on vehicle age, accident history, and mileage
RiskScore	Numeric score: (age > 5 ? 1 : 0) + accidentsInPast + (distanceDriven > 100,000 ? 1 : 0)
Total Loss Rule	If repair cost > 75% of IDV, payout = full IDV; policy becomes INACTIVE
Deductible	Amount the customer pays out-of-pocket before insurer covers the remainder
BCrypt	Password hashing algorithm used for secure storage — default strength factor 10
JWT	JSON Web Token — compact, self-contained token for stateless API authentication
HMAC-SHA256	Hash-based Message Authentication Code using SHA-256 — used to verify Razorpay payment signatures
Razorpay	Indian payment gateway for collecting policy premium payments from customers
RazorpayX	Razorpay's payout API for disbursing claim settlements to customer bank accounts
n8n	Self-hosted workflow automation tool used for sending transactional email notifications
Apache POI	Java library for generating Microsoft Excel (.xlsx) files

OpenPDF	Java PDF generation library (fork of iText 4) for creating invoice and policy PDFs
CORS	Cross-Origin Resource Sharing — HTTP mechanism controlling which origins can access the backend API
CSRF	Cross-Site Request Forgery — disabled since API is stateless (no cookies/sessions)
RC Book	Registration Certificate — official Indian government document proving vehicle ownership
IFSC	Indian Financial System Code — 11-character code identifying a bank branch for NEFT/RTGS
VehicleType	Enum: CAR, EV_CAR, TWO_WHEELER, EV_TWO_WHEELER, AUTO, EV_AUTO, HEAVY_VEHICLE
TransmissionType	Enum: MANUAL, AUTOMATIC
PaymentStatus	Enum: PENDING, PAID, FAILED
MOCK_SUCCESS	Fallback payment status used when RazorpayX API is unavailable (test/dev environment)
WillowShield	The Spring Boot backend Maven module/artifact name
VehicleSure	The commercial product name of the full platform
IRDAI	Insurance Regulatory and Development Authority of India — regulatory body for the insurance sector
VAHAN	India's national vehicle registration database maintained by MoRTH
Fund Account	RazorpayX concept — a verified bank account linked to a contact, used for payouts
Workload DTO	Data Transfer Object showing active assignment count per staff member, used for load balancing

— End of VehicleSure SRS Document —